

DATE LIMITE DE REMISE : 15 décembre 2025, 23h59.

Les étudiantes et étudiants inscrits au cours IFT712 sont tenus de faire un projet de session en python en équipe de 3 ou 4 (obligatoire). Le projet a pour objectif de tester au moins six méthodes de classification sur une base de données Kaggle (www.kaggle.com) avec la bibliothèque scikit-learn (<https://scikit-learn.org>). Les équipes sont libres de choisir la base de données de leur choix, mais une option simple est celle du challenge de classification de feuilles d'arbres (www.kaggle.com/c/leaf-classification). Pour ce projet, on s'attend à ce que les bonnes pratiques de cross-validation et de recherche d'hyper-paramètres soient mises de l'avant pour identifier la meilleure solution possible pour résoudre le problème.

Le barème de correction est le suivant :

Qualité du code - Commentaires	/10
Choix de design	/10
Gestion de projet (Git)	/10
Rapport	/70
... Démarche scientifique	/50
... Analyse des résultats	/20

Qualité du code et commentaires : on vous demande du code rédigé le plus possible soit en français, soit en anglais (mais pas les deux en même temps) Votre code doit aussi être bien documenté, respecter le standard pep8 (<https://www.python.org/dev/peps/pep-0008/>) et respecter un standard uniforme pour la nomenclature des variables, des noms de fonctions et des noms de classes. Évitez également les variables « hardcodées » empêchant l'utilisation de votre programme sur un autre ordinateur que le vôtre.

Choix de design : vous devez organiser votre code de façon professionnelle. Pour ce faire, on s'attend à une hiérarchie de classes cohérente, pas seulement une panoplie de fonctions disparates. Aussi, du code dans un script « qui fait tout » se verra automatiquement attribuer la note de zéro. Bien que non requis, on vous encourage à faire un design de classes avant de commencer à coder et à présenter un diagramme de classe dans votre rapport. Aussi, le code, les données et la documentation doivent être organisés suivant une bonne structure de répertoires. Pour vous aider, vous pouvez utiliser le projet « cookiecutter » (<https://github.com/audreyr/cookiecutter>). La solution proposée doit aussi être facile à utiliser. Bien que non requis, on vous encourage à présenter votre solution sous forme de jupyter notebook(s).

Gestion de projet : comme tout projet qui se respecte, vous devez utiliser un gestionnaire de version de code. On vous demande d'utiliser « git » via la plateforme « GitHub » (incluez votre lien dans votre rapport). On s'attend également à ce que vous fassiez une bonne utilisation de git. Par exemple : évitez de « pousser » du code dans le master sans merge, éviter les « méga » commits, etc. Bien que non requis, on vous encourage aussi à utiliser Trello pour gérer votre projet à haut niveau.

Démarche scientifique : pour ce volet, vous devez vous poser les questions suivantes : avez-vous

bien « cross-validé » vos méthodes? Avez-vous bien fait votre recherche d'hyper-paramètres? Avez-vous entraîné et testé vos méthodes sur les mêmes données? Est-ce que cela transparait dans le rapport? Avez-vous uniquement utilisé les données brutes ou avez-vous essayé de les réorganiser pour améliorer vos résultats? Etc.

Analyse des résultats : que vos résultats soient bons ou non, vous devez en produire une analyse cohérente, potentiellement en les comparant aux résultats de tests déjà présents en ligne sur le site du challenge.

NOTE IMPORTANTE : Veuillez noter que jusqu'à 30 % des points peuvent être retirés pour toute faute aberrante. Par exemple, une équipe qui se refuserait d'utiliser git pourrait perdre jusqu'à 40 points (10 pour la gestion de projet et 30 pour faute aberrante). Autre exemple : ne pas évaluer ses performances sur un ensemble test.